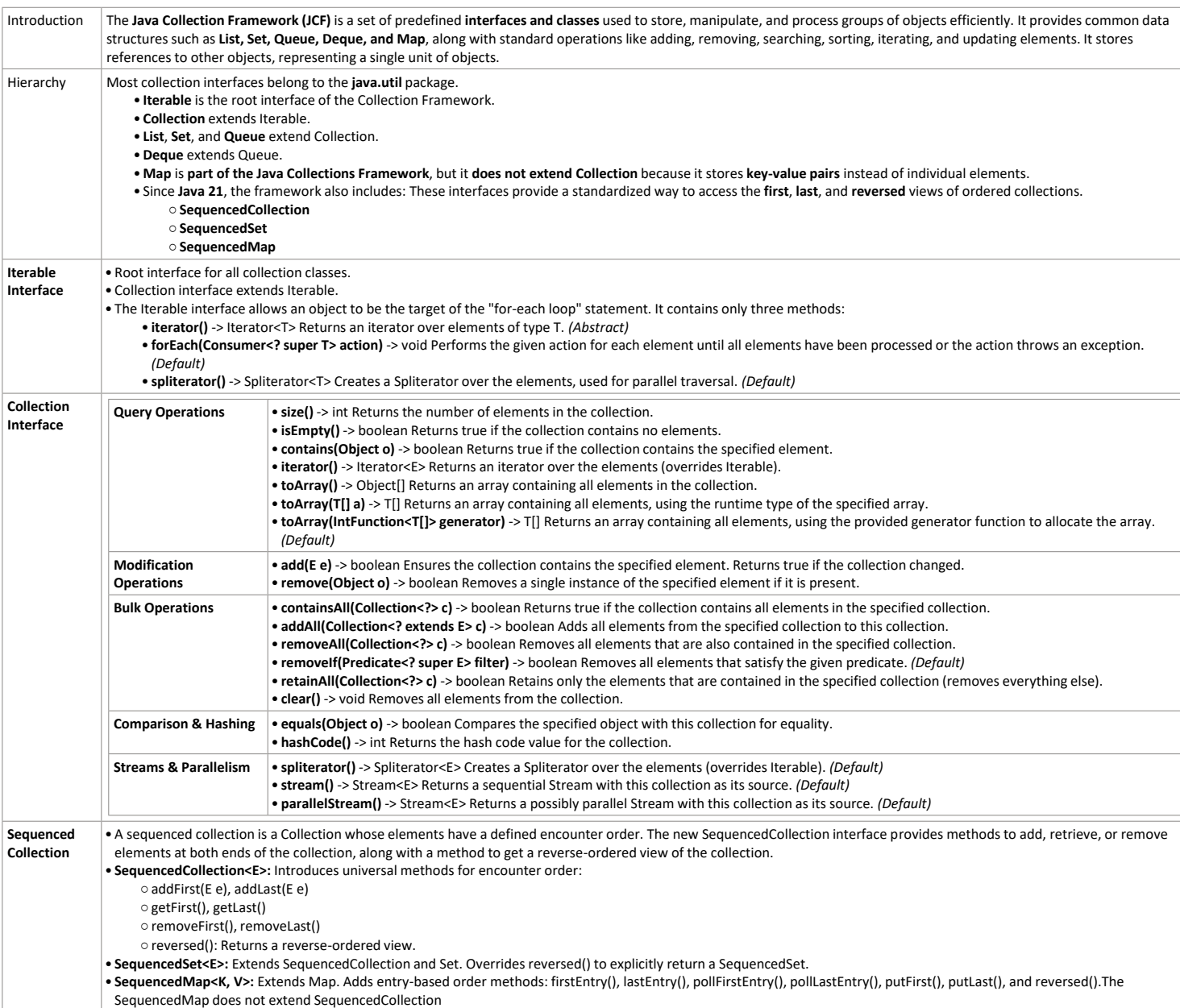


29 July 2018 12:30

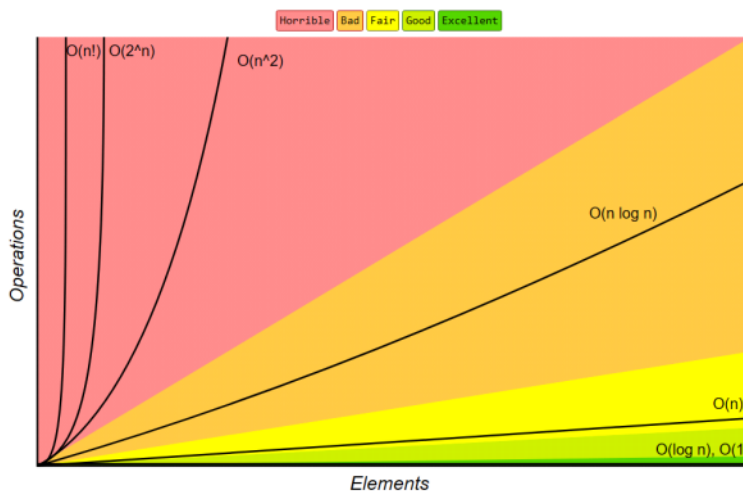


Big O

28 February 2024 08:34

Omega (Ω) – Best Case
 Theta (Θ) - Average Case
 Omicron-Big O (O) - Worst Case

Notation	Name	Description	Example
$O(1)$	Constant Time	Doesn't depend on the size of the data set.	Accessing an array element by its index.
$O(\log n)$	Logarithmic Time: Divide and Conquer	Splits the data in each step (divide and conquer).	Binary search.
$O(n)$	Linear Time: Proportional	Directly proportional to the data set size.	Looping through an array.
$O(n \log n)$	Linearithmic Time	Splits and sorts or searches data.	Merge sort, quick sort.
$O(n^2)$	Polynomial Time: Loop within Loop	Nested loops for each power of n.	Bubble sort ($O(n^2)$).



Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Stack	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Queue	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Singly-Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Doubly-Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Skip List	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log n)$
Hash Table	N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Binary Search Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Cartesian Tree	N/A	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
B-Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
Red-Black Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
Splay Tree	N/A	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	N/A	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
AVL Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
KD Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(\log(n))$
Mergesort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Tree Sort	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(n)$
Shell Sort	$O(n \log(n))$	$O(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$
Cubesort	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$

Pointers

27 March 2024 10:59

```
int num1 = 11;
int num2 = num1;
System.out.println(num1); // 11
System.out.println(num2); // 11
```

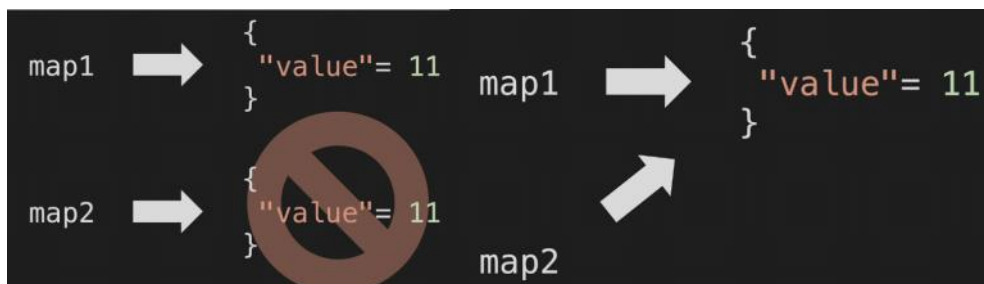
```
num1 = 22;
System.out.println(num1); // 22
System.out.println(num2); // 11
```

```
Map<String, Integer> hm1 = new HashMap<>();
Map<String, Integer> hm2 = new HashMap<>();
hm1.put("Value", 1);
hm2 = hm1;
```

```
System.out.println(hm1); // {Value=1}
System.out.println(hm2); // {Value=1}
```

```
hm1.put("Value", 2); // or hm2.put("Value", 2);
```

```
System.out.println(hm1); // {Value=2}
System.out.println(hm2); // {Value=2}
```



- In the non-pointer example, `num1` and `num2` are independent variables. Changing `num1` does not affect `num2`.
- In the pointer example, `hm1` and `hm2` reference the same `HashMap` object. Changes to the map via `hm1` affect `hm2` and vice versa.

List Interface

02 June 2021 09:34

List (Extends SequencedCollection): A sequential, ordered collection where elements are accessed via a zero-based integer index. Duplicates are permitted.

Internal Structure

- **ArrayList:** Backed by a resizable dynamic array.
- **LinkedList:** Structured as a doubly-linked list where each node points to the previous and next elements.
- **Vector:** Legacy dynamic array implementation with internally synchronized methods.

Performance

- **ArrayList:** Elements are stored in contiguous memory blocks. Modern CPUs leverage spatial locality by loading chunks of adjacent memory into the ultra-fast CPU Cache (L1/L2/L3).
- **LinkedList:** Nodes are allocated dynamically via the new keyword, scattering them across random addresses in the JVM Heap. This results in rampant CPU cache misses during iteration, forcing the CPU to fetch data from slower RAM.
- **Traversal Performance:** Because of CPU cache locality and direct index access, iterating through an ArrayList using a standard loop or stream is orders of magnitude faster than stepping through a LinkedList pointer chain.

Common Methods

- **add(E element):** Adds an element to the end of the list.
- **add(int index, E element):** Inserts an element at the specified index.
- **remove(int index):** Removes the element at the specified index.
- **get(int index):** Returns the element at the specified index.
- **set(int index, E element):** Replaces the element at the specified index with the given element.
- **size():** Returns the number of elements in the list.
- **clear():** Removes all the elements from the list.

ArrayList	LinkedList
• Uses dynamic array. • Maintains insertion order, non-synchronized. • Allows random access. • ArrayList is not efficient for manipulation because too much shifting is required. • ArrayList is better to store and fetch data. • ArrayList takes less memory overhead as it stores only object	• Uses doubly linked list. • Maintains insertion order, non-synchronized. • Does not provide random access. • Fast manipulation due to no shifting. • Not preferred to store and fetch data • LinkedList takes more memory overhead, as it stores the object as well as the address of that object

ArrayList

17 June 2018 11:04

Introduction	The ArrayList class in Java is an implementation of the List interface that employs a dynamic array to store elements. Unlike traditional arrays, it does not have a fixed size, allowing for the flexible addition and removal of elements at any time						
Duplicates & Order	Can contain duplicate elements and maintains their insertion order.						
Synchronization	The class is unsynchronized, meaning it is not thread-safe.						
Automatic Resizing	When the array becomes full, the ArrayList automatically grows. It allocates a new, larger array, copies the old elements over using System.arraycopy(), and drops the old array reference						
Growth Formula	The initial virtual capacity is 10. When this capacity is reached, a new array is created using the formula $\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$. This means it grows by 50% (e.g., 10 → 15 elements, 15 → 22 elements, 22 → 33 elements).						
Capacity Customization	- ArrayList<String> list2 = new ArrayList<String>(20); // Initial Capacity = 20 - ArrayList<String> list = new ArrayList<String>(); list.ensureCapacity(20);						
Optimizing Memory	The trimToSize() method can be used to trim the capacity of the ArrayList down to its current size.						
Positional Access	- add(int index, Object o) - addAll(int index, Collection c) - remove(int index) - get(int index) - set(int index, Object new)						
Search Methods	- indexOf(Object o) - lastIndexOf(Object o)						
Sorting Arrays	- Arrays.sort(arr) or Arrays.parallelSort(largeArray); - Collections.sort(array, Collections.reverseOrder()); - Arr.sort(Comparator.naturalOrder()); // arr is the ArrayList name - Arr.sort(Comparator.reverseOrder());						
Immutable List	An immutable list is a list whose contents cannot be modified after it is created. You cannot add, remove, or replace elements in it. List<String> fruits = List.of("Apple", "Banana", "Orange");						
Iterator (Forward-Only)	The Iterator interface allows you to move forward through any collection and safely remove elements during the loop. <pre>Iterator<String> itr = list.iterator(); // Loop through the list while (itr.hasNext()) { String fruit = itr.next(); System.out.println(fruit); // Safely remove an item during iteration if (fruit.equals("Banana")) { itr.remove(); } }</pre> boolean hasNext(): Returns true if there are more elements in the collection left to traverse. E next(): Returns the next element in the collection and advances the cursor position by one. (Throws a NoSuchElementException if no more elements exist). void remove(): Removes the last element returned by the most recent next() call from the underlying collection. You can only call this once per next() invocation. void forEachRemaining(Consumer<? super E> action): Performs the given action (like a lambda expression) on every remaining element until all items are processed or an exception is thrown.						
ListIterator (Bi-directional)	ListIterator extends Iterator. It only works on List implementations (like ArrayList). It allows you to move both forward and backward, modify elements, and add new items mid-loop. <pre>// Get the list iterator ListIterator<String> litr = list.listIterator(); System.out.println("Forward Traversal:"); while (litr.hasNext()) { System.out.println("Item: " + litr.next() + " Next Index: " + litr.nextIndex()); } System.out.println("\nBackward Traversal:"); while (litr.hasPrevious()) { System.out.println("Item: " + litr.previous() + " Prev Index: " + litr.previousIndex()); }</pre> Forward Traversal boolean hasNext(): Returns true if there are more elements ahead when moving forward. E next(): Returns the next element in the list and moves the cursor forward. int nextIndex(): Returns the index of the element that would be returned by a subsequent call to next(). Returns the list size if the cursor is at the end. Backward Traversal boolean hasPrevious(): Returns true if there are more elements behind when moving in the reverse direction. E previous(): Returns the previous element in the list and moves the cursor backward. int previousIndex(): Returns the index of the element that would be returned by a subsequent call to previous(). Returns -1 if the cursor is at the beginning. Modification Operations void remove(): Removes the last element returned by the last call to either next() or previous(). void set(E e): Replaces the last element returned by next() or previous() with the specified element e (useful for updating items). void add(E e): Inserts the specified element e into the list immediately before the element that would be returned by next().						
Synchronized ArrayList	- Better suited for write-heavy scenarios. - Locks the entire list during read and write operations. - Can be less efficient for frequent read operations. - Requires careful synchronization to avoid deadlocks and other concurrency issues. <table><tr><td>Creation</td><td>List<String> syncList = Collections.synchronizedList(new ArrayList<String>());</td></tr><tr><td>Adding Elements</td><td>syncList.add("Saurav");</td></tr><tr><td>Get 1 Element</td><td>String element = syncList.get(index);</td></tr></table>	Creation	List<String> syncList = Collections.synchronizedList(new ArrayList<String>());	Adding Elements	syncList.add("Saurav");	Get 1 Element	String element = syncList.get(index);
Creation	List<String> syncList = Collections.synchronizedList(new ArrayList<String>());						
Adding Elements	syncList.add("Saurav");						
Get 1 Element	String element = syncList.get(index);						

	<table> <tr> <td>Iterating</td><td> <pre>for (String element : synchronizedList) { System.out.println(element); }</pre> </td></tr> <tr> <td>Updating</td><td> <pre>synchronized (synchronizedList) { synchronizedList.set(index, newValue); }</pre> </td></tr> <tr> <td>Removing data</td><td> <pre>synchronized (synchronizedList) { synchronizedList.remove("Cherry"); }</pre> </td></tr> </table>	Iterating	<pre>for (String element : synchronizedList) { System.out.println(element); }</pre>	Updating	<pre>synchronized (synchronizedList) { synchronizedList.set(index, newValue); }</pre>	Removing data	<pre>synchronized (synchronizedList) { synchronizedList.remove("Cherry"); }</pre>
Iterating	<pre>for (String element : synchronizedList) { System.out.println(element); }</pre>						
Updating	<pre>synchronized (synchronizedList) { synchronizedList.set(index, newValue); }</pre>						
Removing data	<pre>synchronized (synchronizedList) { synchronizedList.remove("Cherry"); }</pre>						
CopyOnWriteArrayList	For scenarios where read operations are more frequent than write operations, consider using CopyOnWriteArrayList. It's a thread-safe alternative that creates a copy of the underlying array on each write operation, ensuring thread safety without the overhead of locking the entire list. • Creates a copy of the underlying array on write operations. • More efficient for read-heavy workloads. • Can be less efficient for frequent write operations due to the overhead of copying. <pre>CopyOnWriteArrayList<String> empList=new CopyOnWriteArrayList<>(); empList.add("Tom"); empList.add("Steve"); empList.forEach(System.out::println);//No Sync Method needed, list is threadsafe list.remove("Banana");</pre>						

LinkedList

24 July 2018 21:25

Introduction	Java LinkedList is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element is connected to its neighboring elements using pointers. Node Anatomy: It stores elements using nodes. Each node contains a data part (which stores the element itself) and a reference part (pointers that point to the next and previous nodes). Head and Tail: • The first node is the head (the entry point), which stores the address of the first node and does not have a previous node. • The last node is the tail, which points to null for its next reference. Basic Properties: • It belongs to the java.util package. • It maintains the insertion order of its elements. • It can contain duplicate elements. • It is non-synchronized (not thread-safe). Versatility: A LinkedList can be dynamically used as a list, stack, or queue.
Performance	• Fast Manipulations: Inserting or removing elements from the middle of a standard array/ArrayList is computationally expensive because all elements beyond that point must be shifted. A LinkedList allows for fast manipulation because no shifting is needed. • No Array Overhead: It doesn't require the same strict memory allocation or reallocation steps as an array-based list during resizing operations. • Sequential Access Only: Nodes cannot be accessed directly via index formulas. You must traverse sequentially starting from the head to reach a desired node. • High Memory Footprint: Every item is wrapped in a Node object, making it a memory hog. Each Node consumes object overhead plus two extra 64-bit pointers (one for next, one for prev). For large datasets (e.g., 50,000 elements), it consumes significantly more heap space than an ArrayList. • Data Types: Cannot store primitive types (like int, float, char) directly. You must use their corresponding object wrapper classes.
Methods	• add(element): Adds an element to the end of the LinkedList. • add(index, element): Inserts an element at the specified index in the LinkedList. • get(index): Returns the element at the specified index in the LinkedList. • remove(index): Removes the element at the specified index in the LinkedList. • size(): Returns the number of elements in the LinkedList. • addFirst(E e): Inserts the given element at the beginning of a list. • void addLast(E e): Appends the given element to the end of a list. • E getFirst(): Returns the first element in a list. • E getLast(): Returns the last element in a list. • E peek(): Retrieves the first element of a list. • E peekFirst(): Retrieves the first element of a list or returns null if a list is empty. • E peekLast(): Retrieves the last element of a list or returns null if a list is empty. • E poll(): Retrieves and removes the first element of a list. • E pollFirst(): Retrieves and removes the first element of a list, or returns null if a list is empty. • E pollLast(): Retrieves and removes the last element of a list, or returns null if a list is empty. • E pop(): Pops an element from the stack represented by a list. • void push(E e): Pushes an element onto the stack represented by a list.

	Linked List	Array List
Append	O(1)	O(1)
Remove Last	O(n)	O(1)
Prepend	O(1)	O(n)
Remove First	O(1)	O(n)
Insert	O(n)	O(n)
Remove	O(n)	O(n)
Lookup by Index	O(n)	O(1)
Lookup by Value	O(n)	O(n)

Next has reference of next element

```
head= {
  "value"= 11,
  "next"= {
    "value"= 3,
    "next"= {
      "value"= 23,
      "next"= {
        "value"= 7,
        "next"= {
          "value"= 4,
          "next"= null
        }
      }
    }
  }
}
```


Stack & Queue

02 June 2021 09:47

Stack	A stack is a linear data structure that follows the Last In First Out (LIFO) principle. This means the last element added to the stack will be the first one to be removed. Imagine a stack of plates: you add a plate on top, and it's also the first one you take off. <table><tr><th>Key Operations</th><th>Purpose</th></tr><tr><td>Push</td><td>Purpose: Adds an element to the top of the stack. Example: <code>stack.push(element)</code></td></tr><tr><td>Pop</td><td>Purpose: Removes the element from the top of the stack. Example: <code>stack.pop()</code></td></tr><tr><td>Peek</td><td>Purpose: Returns the element at the top of the stack without removing it. Example: <code>stack.peek()</code></td></tr><tr><td>isEmpty</td><td>Purpose: Checks if the stack is empty. Example: <code>stack.isEmpty()</code></td></tr><tr><td>size</td><td>Purpose: Returns the number of elements in the stack. Example: <code>stack.size()</code></td></tr></table>	Key Operations	Purpose	Push	Purpose: Adds an element to the top of the stack. Example: <code>stack.push(element)</code>	Pop	Purpose: Removes the element from the top of the stack. Example: <code>stack.pop()</code>	Peek	Purpose: Returns the element at the top of the stack without removing it. Example: <code>stack.peek()</code>	isEmpty	Purpose: Checks if the stack is empty. Example: <code>stack.isEmpty()</code>	size	Purpose: Returns the number of elements in the stack. Example: <code>stack.size()</code>													
Key Operations	Purpose																									
Push	Purpose: Adds an element to the top of the stack. Example: <code>stack.push(element)</code>																									
Pop	Purpose: Removes the element from the top of the stack. Example: <code>stack.pop()</code>																									
Peek	Purpose: Returns the element at the top of the stack without removing it. Example: <code>stack.peek()</code>																									
isEmpty	Purpose: Checks if the stack is empty. Example: <code>stack.isEmpty()</code>																									
size	Purpose: Returns the number of elements in the stack. Example: <code>stack.size()</code>																									
Queue	A queue is a linear data structure that follows the First In First Out (FIFO) principle, where the element added first will be the first one to be removed. Imagine a queue in a supermarket: the first person to join the queue is the first one to be served. Cannot store null but can store duplicates. <table><tr><th>Key Operations</th><th>Example</th></tr><tr><td>Enqueue (Add)</td><td>Purpose: Adds an element to the end of the queue. Example: <code>queue.add(element)</code></td></tr><tr><td>Dequeue (Remove)</td><td>Purpose: Removes the element from the front of the queue. Example: <code>queue.remove()</code></td></tr><tr><td>Peek/Front:</td><td>Purpose: Returns the element at the front of the queue without removing it. Example: <code>queue.peek()</code></td></tr><tr><td>IsEmpty</td><td>Purpose: Checks if the queue is empty. Example: <code>queue.isEmpty()</code></td></tr><tr><td>Size</td><td>Purpose: Returns the number of elements in the queue. Example: <code>queue.size()</code></td></tr></table>	Key Operations	Example	Enqueue (Add)	Purpose: Adds an element to the end of the queue. Example: <code>queue.add(element)</code>	Dequeue (Remove)	Purpose: Removes the element from the front of the queue. Example: <code>queue.remove()</code>	Peek/Front:	Purpose: Returns the element at the front of the queue without removing it. Example: <code>queue.peek()</code>	IsEmpty	Purpose: Checks if the queue is empty. Example: <code>queue.isEmpty()</code>	Size	Purpose: Returns the number of elements in the queue. Example: <code>queue.size()</code>													
Key Operations	Example																									
Enqueue (Add)	Purpose: Adds an element to the end of the queue. Example: <code>queue.add(element)</code>																									
Dequeue (Remove)	Purpose: Removes the element from the front of the queue. Example: <code>queue.remove()</code>																									
Peek/Front:	Purpose: Returns the element at the front of the queue without removing it. Example: <code>queue.peek()</code>																									
IsEmpty	Purpose: Checks if the queue is empty. Example: <code>queue.isEmpty()</code>																									
Size	Purpose: Returns the number of elements in the queue. Example: <code>queue.size()</code>																									
Deque	Deque interface extends the Queue interface. In Deque, we can remove and add the elements from both the side. Deque stands for a double-ended queue which enables us to perform the operations at both the ends. Allows both LIFO and FIFO. Can store null and duplicates <table><tr><th>Operation</th><th></th><th>Throws an exception if operation fails.</th><th>Returns null or false if operation fails.</th></tr><tr><td rowspan="2">Insertion</td><td>Front End</td><td><code>addFirst()</code></td><td><code>offerFirst()</code></td></tr><tr><td>Rear End</td><td><code>addLast()</code></td><td><code>offerLast()</code></td></tr><tr><td rowspan="2">Retrieval</td><td>Front End</td><td><code>getFirst()</code></td><td><code>peekFirst()</code></td></tr><tr><td>Rear End</td><td><code>getLast()</code></td><td><code>peekLast()</code></td></tr><tr><td rowspan="2">Retrieval And Removal</td><td>Front End</td><td><code>removeFirst()</code></td><td><code>pollFirst()</code></td></tr><tr><td>Rear End</td><td><code>removeLast()</code></td><td><code>pollLast()</code></td></tr></table>	Operation		Throws an exception if operation fails.	Returns null or false if operation fails.	Insertion	Front End	<code>addFirst()</code>	<code>offerFirst()</code>	Rear End	<code>addLast()</code>	<code>offerLast()</code>	Retrieval	Front End	<code>getFirst()</code>	<code>peekFirst()</code>	Rear End	<code>getLast()</code>	<code>peekLast()</code>	Retrieval And Removal	Front End	<code>removeFirst()</code>	<code>pollFirst()</code>	Rear End	<code>removeLast()</code>	<code>pollLast()</code>
Operation		Throws an exception if operation fails.	Returns null or false if operation fails.																							
Insertion	Front End	<code>addFirst()</code>	<code>offerFirst()</code>																							
	Rear End	<code>addLast()</code>	<code>offerLast()</code>																							
Retrieval	Front End	<code>getFirst()</code>	<code>peekFirst()</code>																							
	Rear End	<code>getLast()</code>	<code>peekLast()</code>																							
Retrieval And Removal	Front End	<code>removeFirst()</code>	<code>pollFirst()</code>																							
	Rear End	<code>removeLast()</code>	<code>pollLast()</code>																							
PriorityQueue	Elements are ordered according to their natural ordering or by a Comparator provided at queue construction time. PriorityQueue does not permit null elements. it is not synchronized Based on Complete Binary Tree Elements with higher priority are dequeued before elements with lower priority. In default PQ, smallest element will be removed 1st • <code>PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());</code> • <code>boolean add(E e)</code>: Inserts the specified element into this priority queue. • <code>boolean offer(E e)</code>: Inserts the specified element into this priority queue. Does not throw an exception if the insertion fails due to capacity restrictions, making it a safer option when you're not sure about the capacity. • <code>E poll()</code>: Retrieves and removes the head of this queue, or returns null if this queue is empty. • <code>E peek()</code>: Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty. • <code>int size()</code>: Returns the number of elements in this priority queue. • <code>void clear()</code>: Removes all of the elements from this priority queue. • <code>Boolean remove(E e)</code>: Removes a specific element from the queue.																									

Feature	Stack	Queue	Deque (Double-ended Queue)
Data Structure Type	Linear	Linear	Linear
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)	Allows insertion and deletion from both ends
Basic Operations	<code>push()</code> (add to top), <code>pop()</code> (remove from top)	<code>enqueue()</code> (add to rear), <code>dequeue()</code> (remove from front)	Insertions: <code>enqueueFront()</code> , <code>enqueueRear()</code> ; Removals: <code>dequeueFront()</code> , <code>dequeueRear()</code>
Insertion	At the top	At the rear	At both front and rear
Deletion	From the top	From the front	From both front and rear
Use Cases	Undo mechanisms, browser history, recursive	Order processing, scheduling, printers, BFS	Caching, undo-redo, palindromic checks

	function calls		
Direct Access	Not allowed	Not allowed	Not allowed
Storage	Typically array-based or linked list	Typically array-based or linked list	Typically array-based or linked list

Set Interface

02 June 2021 10:39

Set	Present in java.util package. It extends the Collection interface. It represents the unordered set of elements which doesn't allow us to store the duplicate items. We can store at most one null value in Set. Stores Unique Element and it will grow dynamically when elements are added to it.
Initial Capacity and Load Factor	<pre>Set<String> set = new HashSet<>(initialCapacity, loadFactor);</pre> Initial Capacity • What it is: The number of buckets (slots) in the underlying hash table when the set is created. • Default value: 16 • Why it matters: If the capacity is too low, the set will constantly have to resize itself as it grows, which is a slow operation. If it's too high, you waste memory. Load Factor • What it is: A measure of how full the set is allowed to get before its capacity is automatically doubled (resized). • Default value: 0.75 (75% full) • Why it matters: ◦ A higher load factor (e.g., 0.90) decreases memory overhead but increases lookup time because it allows more elements to crowd into fewer buckets (causing hash collisions). ◦ A lower load factor (e.g., 0.50) keeps lookups extremely fast but uses more memory because the set resizes much earlier. Resizing (or "rehashing") happens when the number of elements in the set exceeds the product of the capacity and the load factor. Threshold = Capacity × Load Factor Using the default constructor: Threshold = 16 × 0.75 = 12 Once you add the 13th element, the HashSet will automatically double its capacity to 32 and re-arrange all existing elements into the new buckets.
HashSet	Implements Set Interface. It represents the collection that uses a hash table for storage. Hashing is used to store the elements in the HashSet and contains unique items. • HashSet stores the elements by using a mechanism called hashing. • HashSet contains unique elements only. • HashSet allows 1 null value. • HashSet class is non synchronized. • HashSet doesn't maintain the insertion order. Elements are inserted on the basis of their hashCode. • HashSet is the best approach for search operations. • The initial default capacity of HashSet is 16, and the load factor is 0.75. Size doubles after that and rehashing occurs. Methods in HashSet: • boolean add(E e): Used to add the specified element if it is not present, if it is present then return false. • void clear(): Used to remove all the elements from set. • boolean contains(Object o): Used to return true if an element is present in set. • boolean remove(Object o): Used to remove the element if it is present in set. • Iterator iterator(): Used to return an iterator over the element in the set. • boolean isEmpty(): Used to check whether the set is empty or not. Returns true for empty and false for non-empty condition for set. • int size(): Used to return the size of the set. • Object clone(): Used to create a shallow copy of the set. HashSet uses HashMap internally to store its objects. Whenever you create a HashSet object, one HashMap object associated with it is also created. This HashMap object is used to store the elements you enter in the HashSet. The elements you add into HashSet are stored as keys of this HashMap object. The value associated with those keys will be a constant and the values will be a constant called PRESENT. As we know in a HashMap each key is unique and when we call put(Key, Value) method, it returns the previous value associated with key, or null if there was no mapping for key. So in add() method we check the return value of map.put(key, value) method with null value.
LinkedHashSet	Represents the LinkedList implementation of Set Interface. It extends the HashSet class and implements Set interface. Like HashSet, It also contains unique elements. It maintains the insertion order and permits null elements. The insertion order of elements into LinkedHashMap are maintained by adding two new fields to this class. They are before and after. These two fields hold the references to previous and next elements. These two fields make LinkedHashMap to function as a doubly linked list. Java LinkedHashSet class is a Hashtable and doubly Linked list implementation of the set interface. It inherits HashSet class and implements Set interface. • Java LinkedHashSet class contains unique elements only like HashSet. • Allows null element. • Java LinkedHashSet class provides all optional set operation and permits null elements. • Java LinkedHashSet class is non synchronized. • Java LinkedHashSet class maintains insertion order.
SortedSet	Alternate of Set interface that provides a total ordering on its elements. The elements of the SortedSet are arranged in the increasing (ascending) order. The SortedSet provides the additional methods that inhibit the natural ordering of the elements.
TreeSet	Implements the Set interface and sorted set interface that uses a tree for storage. Like HashSet, TreeSet also contains unique elements. However, the access and retrieval time of TreeSet is quite fast logn. The elements in TreeSet stored in ascending order. • Java TreeSet class contains unique elements only like HashSet. • Java TreeSet class access and retrieval times are quite fast. • Java TreeSet class doesn't allow null element. • Java TreeSet class is non synchronized. • Java TreeSet class maintains ascending order.

Map Interface

12 February 2019 22:38

Introduction	A Map in Java stores key-value pairs, with each pair known as an entry. It is a parameterized abstract data type containing two generic variables, Map<K, V>, where K is the key type and V is the value type. Maps are ideal for searching, updating, or deleting elements based on a key. Map belongs to the Java Collections Framework but does not extend the Collection interface. Key Characteristics of Map Interface: • Uniqueness: Each key is unique and maps to at most one value. • Null Handling: HashMap and LinkedHashMap allow null keys and values; TreeMap does not. • Order: The order of entries depends on the implementation (TreeMap and LinkedHashMap have predictable orders, HashMap does not). • Interfaces and Classes: Implemented via Map and SortedMap interfaces, and HashMap, TreeMap, and LinkedHashMap classes. • Traversal: Maps cannot be directly traversed; use keySet() or entrySet() to convert them to a Set.	
Common Methods	Method	Action Performed
	clear()	This method is used in Java Map Interface to clear and remove all of the elements or mappings from a specified Map collection.
	containsKey(Object)	This method is used in Map Interface in Java to check whether a particular key is being mapped into the Map or not. It takes the key element as a parameter and returns True if that element is mapped in the map.
	containsValue(Object)	This method is used in Map Interface to check whether a particular value is being mapped by a single or more than one key in the Map. It takes the value as a parameter and returns True if that value is mapped by any of the keys in the map.
	entrySet()	This method is used in Map Interface in Java to create a set out of the same elements contained in the map. It basically returns a set view of the map or we can create a new set and store the map elements into them.
	equals(Object)	This method is used in Java Map Interface to check for equality between two maps. It verifies whether the elements of one map passed as a parameter is equal to the elements of this map or not.
	get(Object)	This method is used to retrieve or fetch the value mapped by a particular key mentioned in the parameter. It returns NULL when the map contains no such mapping for the key.
	hashCode()	This method is used in Map Interface to generate a hashCode for the given map containing keys and values.
	isEmpty()	This method is used to check if a map is having any entry for key and value pairs. If no mapping exists, then this returns true.
	keySet()	This method is used in Map Interface to return a Set view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa.
	put(Object, Object)	This method is used in Java Map Interface to associate the specified value with the specified key in this map.
	putAll(Map)	This method is used in Map Interface in Java to copy all of the mappings from the specified map to this map.
	remove(Object)	This method is used in Map Interface to remove the mapping for a key from this map if it is present in the map.
	size()	This method is used to return the number of key/value pairs available in the map.
	values()	This method is used in Java Map Interface to create a collection out of the values of the map. It basically returns a Collection view of the values in the HashMap.
getOrDefault(Object key, V defaultValue)	Returns the value to which the specified key is mapped, or defaultValue if this map contains no mapping for the key.	
Immutable Map	Map<Integer, String> dynamicMap = Map.ofEntries(entry(1, "Tom"),entry(2, "Dick"),entry(3, "Harry"));	
HashMap	HashMap is a part of the Java collection framework. It uses a technique called Hashing. It implements the map interface. It stores the data in the pair of Key and Value. HashMap contains an array of the nodes, and the node is represented as a class. It uses an array and LinkedList data structure internally for storing Key and Value. • Key-Value Storage: Contains values based on the key. • Unique Keys: Only unique keys are allowed. • Null Handling: May have one null key and multiple null values. • Synchronization: Non-synchronized. • Order: Does not maintain any order. • Capacity and Load Factor • Default initial capacity: 16. • Default load factor: 0.75. • Threshold: Increases capacity from 16 to 32 after the 12th entry.The capacity of the HashMap is doubled each time it reaches the threshold. i.e the capacity is increased to 25=32, 26=64, 27=128..... when the threshold is reached. • Hashing : A technique of converting a large String to a small String that represents the same String. A shorter value helps in indexing and faster searches HashMap uses keys in the same way as an Array uses an index	
Iterating in Map	Map.forEach	<pre>map.forEach((key, value) -> { System.out.println(key + " => " + value); });</pre>
	map.entrySet()	<pre>for (Map.Entry<String, Integer> entry : map.entrySet()) { System.out.println(entry.getKey() + " => " + entry.getValue()); }</pre>
	Stream API	<pre>map.entrySet().stream().forEach(entry -> { System.out.println(entry.getKey() + " => " + entry.getValue()); });</pre>
	Iterator with map.entrySet()	<pre>Iterator<Map.Entry<String, Integer>> iterator = map.entrySet().iterator(); while (iterator.hasNext()) { Map.Entry<String, Integer> entry = iterator.next(); System.out.println(entry.getKey() + " => " + entry.getValue()); // safe removal if needed: // iterator.remove(); }</pre>
	Iterating Keys or Values Separately	<pre>// Keys only for (String key : map.keySet()) { System.out.println("Key: " + key); } // Values only for (Integer value : map.values()) { System.out.println("Value: " + value); }</pre>

Synchronize HashMap	Map<String, String> sMap = Collections.synchronizedMap(hMap);																												
Hashtable	Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface. • A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the hashCode() method. A Hashtable contains values based on the key. • Java Hashtable class contains unique elements. • Java Hashtable class doesn't allow null key or value. • Java Hashtable class is synchronized. • The initial default capacity of Hashtable class is 11 whereas loadFactor is 0.75. Hashtable does: public synchronized V put(...) meaning: ONE THREAD AT A TIME FOR THE WHOLE TABLE																												
LinkedHashMap	It is just like HashMap with the additional feature of maintaining an order of elements inserted into it. HashMap provided the advantage of quick insertion, search, and deletion but it never maintained the track and order of insertion which the LinkedHashMap provides where the elements can be accessed in their insertion order.																												
TreeMap	The TreeMap in Java is used to implement the Map interface and NavigableMap along with the Abstract Class. The map is sorted according to the natural ordering of its keys, or by a Comparator provided at map creation time, depending on which constructor is used. This proves to be an efficient way of sorting and storing the key-value pairs. The storing order maintained by the treemap must be consistent with equals just like any other sorted map, irrespective of the explicit comparators																												
ConcurrentHashMap	It is a thread-safe implementation of the Map interface. It allows multiple threads to read and write data simultaneously, without the need for locking the entire map. Unlike a regular HashMap, which is not thread-safe, ConcurrentHashMap ensures that the operations are thread-safe, making it ideal for scenarios where multiple threads need to access and modify the map concurrently. • Provides thread-safe operations without locking the entire map. • Uses Compare And Swap to give thread safety • Supports atomic operations like putIfAbsent(), replace() and remove() • The default concurrency level of ConcurrentHashMap is 16 • Inserting null objects is not possible in ConcurrentHashMap as a key or value. Map Interface ConcurrentMap Class ConcurrentNavigableMap Class Serializable Class ConcurrentHashMap Class extends extends implements implements It implements Serializable, ConcurrentMap<K, V>, Map<K, V> interfaces and extends AbstractMap<K, V> class. Advantages • Thread-safe: ConcurrentHashMap is designed to be used by multiple threads simultaneously, making it an ideal choice for applications that need to handle concurrent access to data. • Fine-grained locking: Unlike other synchronization mechanisms that lock the entire data structure, ConcurrentHashMap uses fine-grained locking to lock only the portion of the map being modified. This makes it highly scalable and efficient for concurrent operations. • Atomic operations: ConcurrentHashMap provides several methods for performing atomic operations, such as putIfAbsent(), replace(), and remove(), which can be useful for implementing complex concurrent algorithms. • High performance: Due to its fine-grained locking mechanism, ConcurrentHashMap is able to achieve high performance, even under heavy concurrent access. Disadvantages • Higher memory overhead: The fine-grained locking mechanism used by ConcurrentHashMap requires additional memory overhead compared to other synchronization mechanisms. • Complexity: The fine-grained locking mechanism used by ConcurrentHashMap can make the code more complex, especially for developers who are not familiar with concurrent programming. ConcurrentHashMap isn't entirely lock-free. It is lock-free for reads (which use volatile memory reads without any locking) and lock-free for inserts into empty buckets (via CAS). It only uses targeted locking when dealing with hash collisions or resizing.																												
Comparison	<table><tr><th>Feature</th><th>HashMap</th><th>Hashtable</th><th>ConcurrentHashMap</th></tr><tr><td>Thread safe</td><td>No</td><td>Yes</td><td>Yes</td></tr><tr><td>Whole map lock</td><td>No</td><td>Yes</td><td>No</td></tr><tr><td>Concurrent reads</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Concurrent writes</td><td>No</td><td>No</td><td>Partial</td></tr><tr><td>Null key</td><td>Yes</td><td>No</td><td>No</td></tr><tr><td>Performance</td><td>Fast</td><td>Slow</td><td>Fast</td></tr></table>	Feature	HashMap	Hashtable	ConcurrentHashMap	Thread safe	No	Yes	Yes	Whole map lock	No	Yes	No	Concurrent reads	No	No	Yes	Concurrent writes	No	No	Partial	Null key	Yes	No	No	Performance	Fast	Slow	Fast
Feature	HashMap	Hashtable	ConcurrentHashMap																										
Thread safe	No	Yes	Yes																										
Whole map lock	No	Yes	No																										
Concurrent reads	No	No	Yes																										
Concurrent writes	No	No	Partial																										
Null key	Yes	No	No																										
Performance	Fast	Slow	Fast																										

Working of HashMap

03 June 2021 11:39

Internally HashMap contains an array of Node and a node is represented as a class that contains 4 fields. It can be seen that the node contains a reference to its object. So it's a linked list.

1. int hash
2. K key
3. V value
4. Node next:reference to the next node in case of a collision

Node<K,V>
int hash K key V value Node<K,V> next

The whole HashMap data structure is based on the principle of Hashing. Hashing is nothing but the function or algorithm or method which when applied on any object/variable returns a unique integer value representing that object/variable. This unique integer value is called hash code. Hash function or simply hash said to be the best if it returns the same hash code each time it is called on the same object. Two objects can have same hash code.

A hash function transforms an object into an integer hash value. Different objects may produce the same hash value, which is called a collision.

Before understanding the internal working of HashMap, you must be aware of hashCode() and equals() method.

- **equals()**: It checks the equality of two objects. It compares the Key, whether they are equal or not. It is a method of the Object class. It can be overridden. If you override the equals() method, then it is mandatory to override the hashCode() method. HashMap uses equals() to compare the key to whether they are equal or not. If the equals() method return true, they are equal otherwise not equal.
- **hashCode()**: This is the method of the object class. hashCode() method is used to get the hash code of an object. hashCode() method of the object class returns the memory reference of an object in integer form. In HashMap, hashCode() is used to calculate the bucket and therefore calculate the index.
- **Buckets**: Array of the node is called buckets. Each node has a data structure like a LinkedList. More than one node can share the same bucket. It may be different in capacity.

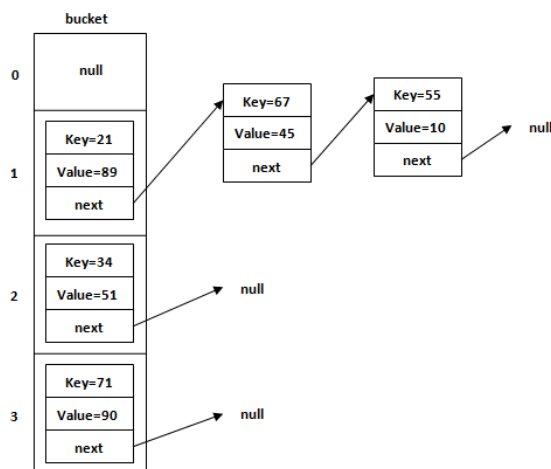


Figure: Allocation of nodes in Bucket

Insert Key, Value pair in HashMap

We use put() method to insert the Key and Value pair in the HashMap. The default size of HashMap is 16 (0 to 15).

```
HashMap<String, Integer> map = new HashMap<>();  
map.put("Aman", 19);  
map.put("Sunny", 29);  
map.put("Ritesh", 39);
```

When we call the put() method, then it calculates the hash code of the Key "Aman." Suppose the hash code of "Aman" is 2657860. To store the Key in memory, we have to calculate the index.

Calculating Index

Index minimizes the size of the array. The Formula for calculating the index is:

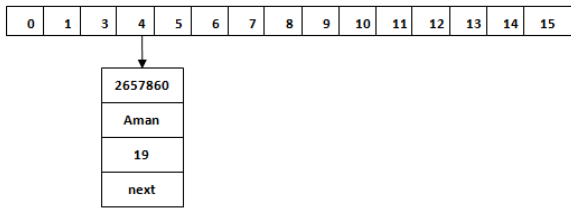
$$\text{index} = (n - 1) \& \text{hash}$$

Where n is the size of the array. Hence the index value for "Aman" is:

$$\text{index} = \text{hashCode}(\text{key}) \& (n - 1)$$

The bitwise AND operation ensures even distribution of entries across buckets.

The value 4 is the computed index value where the Key and value will store in HashMap.

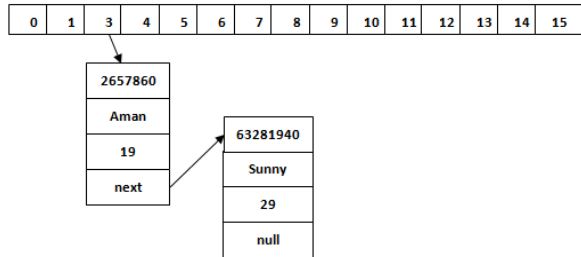


Hash Collision

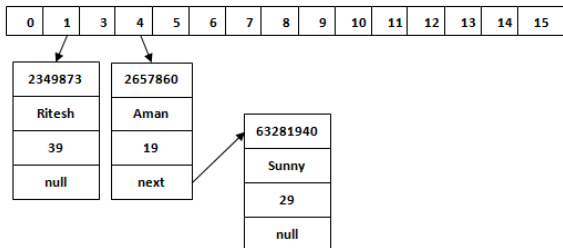
This is the case when the calculated index value is the same for two or more Keys. Let's calculate the hash code for another Key "Sunny." Suppose the hash code for "Sunny" is 63281940. To store the Key in the memory, we have to calculate index by using the index formula.

Index= $63281940 \& (16-1) = 4$

The value 4 is the computed index value where the Key will be stored in HashMap. In this case, equals() method check that both Keys are equal or not. If Keys are same, replace the value with the current value. Otherwise, connect this node object to the existing node object through the LinkedList. Hence both Keys will be stored at index 4.



Similarly, we will store the Key "Ritesh." Suppose hash code for the Key is 2349873. The index value will be 1. Hence this Key will be stored at index 1.



get() method in HashMap

get() method is used to get the value by its Key. It will not fetch the value if you don't know the Key. When get(K Key) method is called, it calculates the hash code of the Key.

Suppose we have to fetch the Key "Aman." The following method will be called.

```
map.get(new Key("Aman"));
```

It generates the hash code as 2657860. Now calculate the index value of 2657860 by using index formula. The index value will be 4, as we have calculated above. get() method search for the index value 4. It compares the first element Key with the given Key. If both keys are equal, then it returns the value else check for the next element in the node if it exists. In our scenario, it is found as the first element of the node and return the value 19.

Let's fetch another Key "Sunny."

The hash code of the Key "Sunny" is 63281940. The calculated index value of 63281940 is 4, as we have calculated for put() method. Go to index 4 of the array and compare the first element's Key with the given Key. It also compares Keys. In our scenario, the given Key is the second Key. It compares the second element Key with the specified Key and returns the value 29. It returns null if the next of the node is null.

To maintain fast retrieval times, Java 8 introduces **TreeNodes**, which form a self-balancing binary tree (specifically a Red-Black tree) inside heavily collided buckets.

- **Treeify Threshold:** When a bucket's linked list reaches **8** elements (TREEIFY_THRESHOLD = 8), the list automatically transforms into a balanced binary tree.
- **Untreeify Threshold:** If elements are later removed or the map resizes, and the bucket size drops down to **6** elements (UNTREEIFY_THRESHOLD = 6), the tree automatically converts back into a plain linked list.

Why not use Trees all the time?

TreeNodes take up roughly **twice the memory size** of regular linked list nodes. Because of this extra memory overhead, Java only deploys them when a bucket contains enough colliding nodes to justify the trade-off. If your keys have well-distributed hash codes, tree bins are rarely needed.

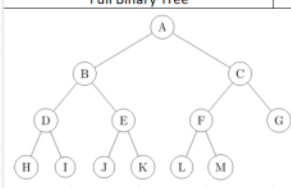
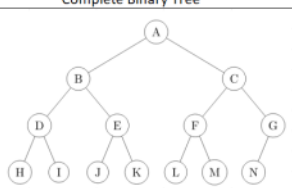
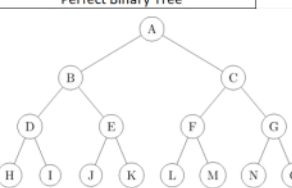
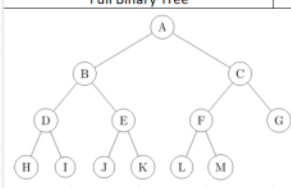
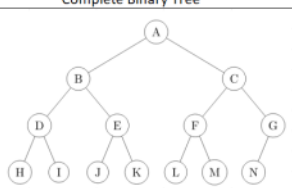
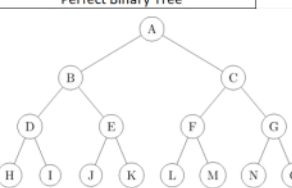
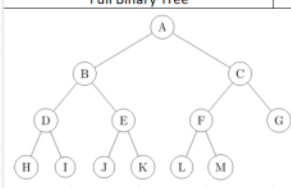
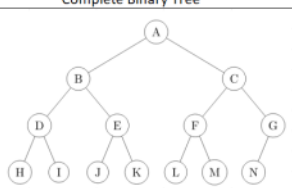
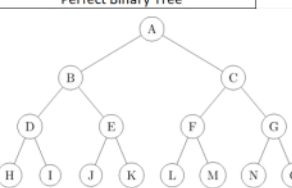
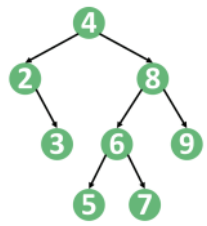
Interview

02 March 2019 08:36

Collection	Collection is a root level interface of the Java Collection Framework. It is a set of classes and interfaces designed for storing and manipulating object data. It includes classes like ArrayList, Vector, Stack, HashSet, and interfaces like List, Queue, and Set.									
Collections	Collections is an utility class in java.util package. It consists of only static methods which are used to operate on objects of type Collection. - Collections.max():This method returns maximum element in the specified collection. - Collections.min():This method returns minimum element in the given collection. - Collections.sort():This method sorts the specified collection. - Collections.copy():This method copies all elements from one collection to another collection. - Collections.reverse():This method reverses the order of elements in the specified collection.									
Array to List	new ArrayList<>(Arrays.asList(array));									
List to Array	String[] array = fruitList.toArray(new String[fruitList.size()]);									
Iterator vs ListIterator	<table><tr><th>Iterator</th><th>ListIterator</th></tr><tr><td>Forward direction only</td><td>Forward and backward direction</td></tr><tr><td>Used in List, Set, Queue</td><td>Used in List only</td></tr><tr><td>Can perform remove operation</td><td>Can perform add, remove, and set operations</td></tr></table>		Iterator	ListIterator	Forward direction only	Forward and backward direction	Used in List, Set, Queue	Used in List only	Can perform remove operation	Can perform add, remove, and set operations
Iterator	ListIterator									
Forward direction only	Forward and backward direction									
Used in List, Set, Queue	Used in List only									
Can perform remove operation	Can perform add, remove, and set operations									
Fail-fast	The Iterator in Java that immediately throws ConcurrentModificationException if any structural modification occurs is called a Fail-fast iterator. It does not require extra space in memory.									
Synchronizing	Collections class provides methods to make List, Set, or Map elements synchronized: - public static List synchronizedList(List l){} - public static Set synchronizedSet(Set s){} - public static SortedSet synchronizedSortedSet(SortedSet s){} - public static Map synchronizedMap(Map m){} - public static SortedMap synchronizedSortedMap(SortedMap m){}									
Length of Array vs. size of ArrayList:	Array length is obtained using the property of length; ArrayList uses size() method.									
Reverse ArrayList	Use Collections.reverse() method. Add example									
Sort ArrayList in descending order	Use Collections.sort() method with Collections.reverseOrder() comparator Add example									
Copying Arrays	• System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length); • src : The source array (where you are copying <i>from</i>). • srcPos : The starting position (index) in the source array. • dest : The destination array (where you are copying <i>to</i>). • destPos : The starting position (index) in the destination array. • length : The number of elements to copy. • int[] newArray = Arrays.copyOf(originalArray, newLength); • int[] newArray = Arrays.copyOfRange(originalArray, fromIndex, toIndex);									

BST

24 November 2024 09:18

Tree	It is a data structure that is used to represent the data in hierarchical form. It can be defined as a collection of objects or entities called as nodes that are linked together to simulate a hierarchy. Tree is a non-linear data structure as the data in a tree is not stored linearly or sequentially. Node without children is called leaf. Full Tree: Every node points to either zero or two nodes. Complete Tree: Nodes are filled from left to right with no gaps. Perfect Tree: All levels are fully filled with nodes. <table><tr><th>Full Binary Tree</th><th>Complete Binary Tree</th><th>Perfect Binary Tree</th></tr><tr><td></td><td></td><td></td></tr></table>					Full Binary Tree	Complete Binary Tree	Perfect Binary Tree																	
Full Binary Tree	Complete Binary Tree	Perfect Binary Tree																							
																									
BST	It follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root. 																								
Space Time Complexity	<table><tr><th>Operations</th><th>Best case time complexity</th><th>Average case time complexity</th><th>Worst case time complexity</th><th>Space complexity</th></tr><tr><td>Insertion</td><td>$O(\log n)$</td><td>$O(\log n)$</td><td>$O(n)$</td><td>$O(n)$</td></tr><tr><td>Deletion</td><td>$O(\log n)$</td><td>$O(\log n)$</td><td>$O(n)$</td><td>$O(n)$</td></tr><tr><td>Search</td><td>$O(\log n)$</td><td>$O(\log n)$</td><td>$O(n)$</td><td>$O(n)$</td></tr></table>	Operations	Best case time complexity	Average case time complexity	Worst case time complexity	Space complexity	Insertion	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	Deletion	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	Search	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$				
Operations	Best case time complexity	Average case time complexity	Worst case time complexity	Space complexity																					
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$																					
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$																					
Search	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$																					